We selected numerical variables that describe countries' health, education, population, and economic conditions. Some values were missing, so we replaced them with the median of each variable. We kept real outliers because countries with unusually high or low values may represent meaningful differences. Finally, we standardized the variables. This was necessary because variables such as GDP and fertility use very different scales.

```
import pandas as pd
from google.colab import files

uploaded = files.upload()
```

Elegir archivos Ningún archivo seleccionado Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
import pandas as pd
df = pd.read_csv("UN35.csv")
df.head()
```

	country	region	tfr	contraception	educationMale	educationFemale	lifeMale	lifeFemale	infantMortality	GDPper
0	Afghanistan	Asia	6.90	NaN	NaN	NaN	45.0	46.0	154.0	
1	Albania	Europe	2.60	NaN	NaN	NaN	68.0	74.0	32.0	
2	Algeria	Africa	3.81	52.0	11.1	9.9	67.5	70.3	44.0	
3	American.Samoa	Asia	NaN	NaN	NaN	NaN	68.0	73.0	11.0	
4	Andorra	Europe	NaN	NaN	NaN	NaN	NaN	NaN	NaN	

Empieza a programar o a crear código con IA.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

```
df.shape
```

```
(207, 14)
```

here we checked the size of data

```
df.info()
df.describe().T
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 207 entries, 0 to 206
Data columns (total 14 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   country                 207 non-null    object
 1   region                  207 non-null    object
 2   tfr                     197 non-null    float64
 3   contraception           144 non-null    float64
 4   educationMale           76 non-null     float64
 5   educationFemale         76 non-null     float64
 6   lifeMale                196 non-null    float64
 7   lifeFemale              196 non-null    float64
 8   infantMortality         201 non-null    float64
 9   GDPperCapita            197 non-null    float64
 10  economicActivityMale    165 non-null    float64
 11  economicActivityFemale  165 non-null    float64
 12  illiteracyMale          160 non-null    float64
 13  illiteracyFemale        160 non-null    float64
dtypes: float64(12), object(2)
memory usage: 22.8+ KB
```

	count	mean	std	min	25%	50%	75%	max
tfr	197.0	3.528782	1.761974	1.19	1.950	3.070	4.980	8.0
contraception	144.0	43.430556	23.706732	2.00	21.000	47.000	64.000	86.0
educationMale	76.0	11.410526	3.049834	3.30	9.750	11.250	13.900	17.2
educationFemale	76.0	11.275000	3.475135	2.00	9.325	11.650	13.650	17.8
lifeMale	196.0	63.625510	9.786870	36.00	57.375	66.500	70.900	77.4
lifeFemale	196.0	68.391837	11.041234	39.10	59.600	72.150	76.425	82.9
infantMortality	201.0	43.477612	38.756041	2.00	12.000	30.000	66.000	169.0
GDPperCapita	197.0	6261.954315	9355.659390	36.00	442.000	1779.000	7272.000	42416.0
economicActivityMale	165.0	76.460606	7.687047	51.20	72.300	76.800	81.200	93.0
economicActivityFemale	165.0	46.794545	16.810472	1.90	37.000	48.400	56.400	90.6
illiteracyMale	160.0	17.554662	17.813000	0.20	2.952	10.829	27.575	79.1
illiteracyFemale	160.0	27.906200	26.580664	0.20	4.847	20.100	48.025	93.4

The dataset contains 207 countries. country and region are categorical identification variables, while the remaining columns contain numerical demographic, social and economic information.

```
missing = pd.DataFrame({
    "Missing values": df.isnull().sum(),
    "Missing percentage": (df.isnull().mean() * 100).round(1)
})

missing.sort_values("Missing percentage", ascending=False)
```

	Missing values	Missing percentage
educationMale	131	63.3
educationFemale	131	63.3
contraception	63	30.4
illiteracyMale	47	22.7
illiteracyFemale	47	22.7
economicActivityMale	42	20.3
economicActivityFemale	42	20.3
lifeFemale	11	5.3
lifeMale	11	5.3
tfr	10	4.8
GDPperCapita	10	4.8
infantMortality	6	2.9
country	0	0.0
region	0	0.0

this checks missing values

```
features = [
    "tfr",
    "contraception",
    "lifeMale",
    "lifeFemale",
    "infantMortality",
    "GDPperCapita",
    "illiteracyMale",
    "illiteracyFemale"
]

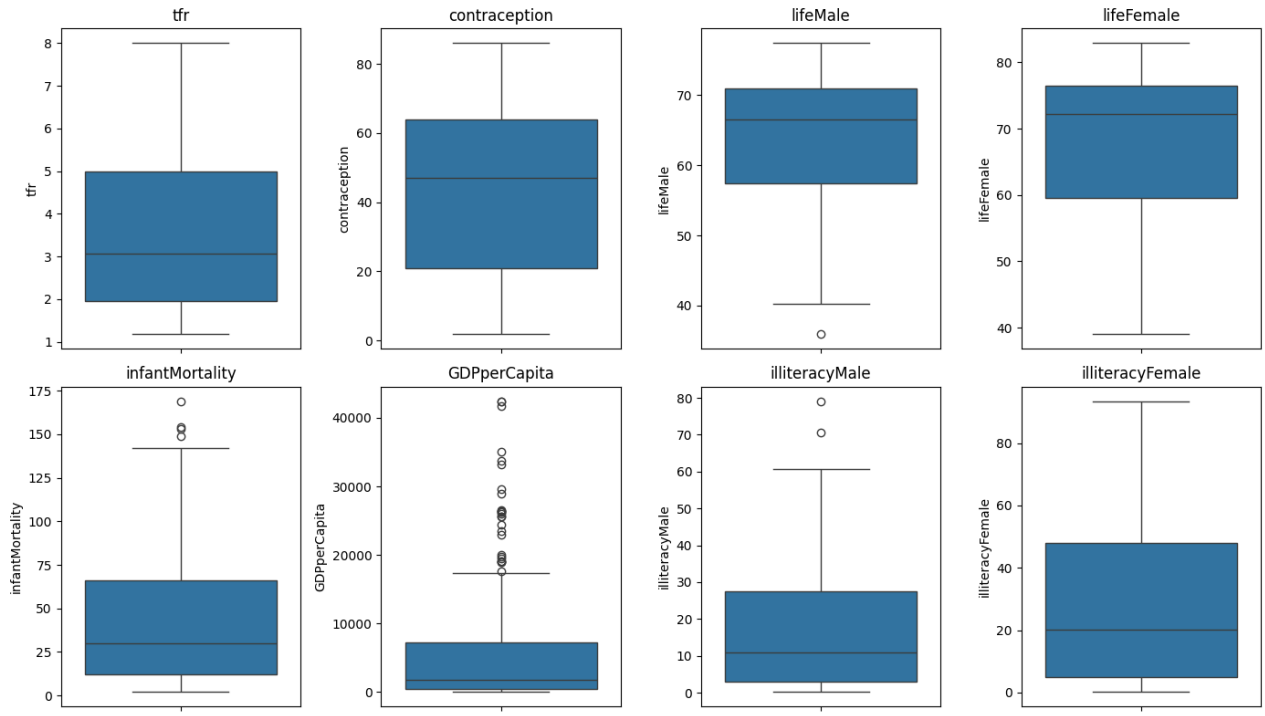
X = df[features].copy()
```

Country and region were excluded because PCA and K-Means require numerical variables. EducationMale and educationFemale were excluded because approximately 63% of their observations are missing. The economic-activity variables were excluded to keep the analysis focused on the main demographic, health, literacy and economic indicators associated with national development.

```
plt.figure(figsize=(14, 8))

for i, column in enumerate(features, 1):
    plt.subplot(2, 4, i)
    sns.boxplot(y=X[column])
    plt.title(column)

plt.tight_layout()
plt.show()
```




```
outlier_counts = {}

for column in features:
    Q1 = X[column].quantile(0.25)
    Q3 = X[column].quantile(0.75)
    IQR = Q3 - Q1
```

```
    outliers = X[
        (X[column] < Q1 - 1.5 * IQR) |
        (X[column] > Q3 + 1.5 * IQR)
    ]

    outlier_counts[column] = len(outliers)

pd.Series(outlier_counts, name="Potential outliers")
```

	Potential outliers
tfr	0
contraception	0
lifeMale	1
lifeFemale	0
infantMortality	4
GDPperCapita	27
illiteracyMale	2
illiteracyFemale	0

dtype: int64

The boxplots and IQR rule identified several potential outliers. These observations were retained because they represent real countries with meaningful demographic or economic characteristics rather than data-entry errors. Removing them could eliminate relevant groups of countries. Their possible influence is considered when interpreting the clusters.

```
X_imputed = X.fillna(X.median())

X_imputed.isnull().sum()
```

	0
tfr	0
contraception	0
lifeMale	0
lifeFemale	0
infantMortality	0
GDPperCapita	0
illiteracyMale	0
illiteracyFemale	0

dtype: int64

Missing numerical values were replaced with the median of each variable. The median is less influenced by extreme values than the mean and allows all countries to remain in the analysis.

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_imputed)

X_scaled = pd.DataFrame(
    X_scaled,
    columns=features,
    index=df.index
)

X_scaled.head()
```

	tfr	contraception	lifeMale	lifeFemale	infantMortality	GDPperCapita	illiteracyMale	illiteracyFemale
0	1.975954	0.125584	-1.972337	-2.101632	2.906362	-0.349277	2.317990	2.502358
1	-0.527921	0.125584	0.443422	0.503134	-0.290522	-0.566115	-0.327698	-0.256491
2	0.176658	0.378463	0.390905	0.158933	0.023925	-0.493144	0.634926	1.057044
3	-0.254241	0.125584	0.443422	0.410107	-0.840806	-0.466053	-0.993675	-1.095624
4	-0.254241	0.125584	0.285872	0.331034	-0.342930	-0.466053	-0.327698	-0.256491

```
pd.DataFrame({
    "Mean": X_scaled.mean(),
    "Standard deviation": X_scaled.std(ddof=0)
}).round(3)
```

	Mean	Standard deviation
tfr	-0.0	1.0
contraception	-0.0	1.0
lifeMale	0.0	1.0
lifeFemale	-0.0	1.0
infantMortality	-0.0	1.0
GDPperCapita	0.0	1.0
illiteracyMale	0.0	1.0
illiteracyFemale	0.0	1.0

Standardization is necessary because the variables use very different scales. For example, GDP per capita has much larger numerical values than fertility or life expectancy. Without standardization, variables with large values would have an excessive influence on PCA and K-Means.

```
pca = PCA()
X_pca_all = pca.fit_transform(X_scaled)

explained_variance = pca.explained_variance_ratio_
cumulative_variance = np.cumsum(explained_variance)

variance_table = pd.DataFrame({
    "Component": range(1, len(features) + 1),
    "Explained variance (%)": explained_variance * 100,
    "Cumulative variance (%)": cumulative_variance * 100
})

variance_table.round(2)
```

	Component	Explained variance (%)	Cumulative variance (%)
0	1	70.23	70.23
1	2	12.57	82.80
2	3	6.63	89.42
3	4	5.93	95.35
4	5	3.14	98.49
5	6	0.91	99.40
6	7	0.47	99.86
7	8	0.14	100.00

```
n_components_90 = np.argmax(cumulative_variance >= 0.90) + 1
n_components_90
```

```
np.int64(4)
```

here we determine the number needed for at least 90%:

```
pca_final = PCA(n_components=n_components_90)
X_pca = pca_final.fit_transform(X_scaled)
```

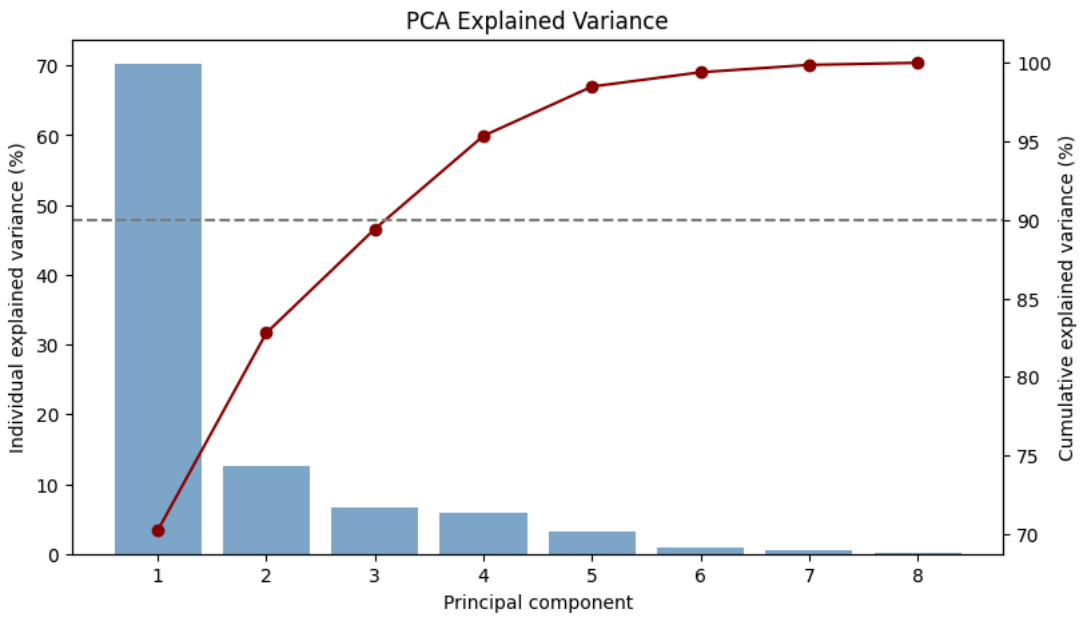
```
fig, ax1 = plt.subplots(figsize=(9, 5))

ax1.bar(
    range(1, len(explained_variance) + 1),
    explained_variance * 100,
    color="steelblue",
    alpha=0.7
)
ax1.set_xlabel("Principal component")
ax1.set_ylabel("Individual explained variance (%)")

ax2 = ax1.twinx()
```

```
ax2.plot(
    range(1, len(cumulative_variance) + 1),
    cumulative_variance * 100,
    color="darkred",
    marker="o"
)
ax2.axhline(90, color="gray", linestyle="--")
ax2.set_ylabel("Cumulative explained variance (%)")

plt.title("PCA Explained Variance")
plt.show()
```




```
loadings = pd.DataFrame(
    pca_final.components_.T,
    index=features,
    columns=[f"PC{i}" for i in range(1, n_components_90 + 1)]
)

loadings.round(3)
```

	PC1	PC2	PC3	PC4
tfr	-0.372	-0.049	-0.070	0.169
contraception	0.307	0.160	0.867	-0.256
lifeMale	0.391	0.153	-0.287	-0.262
lifeFemale	0.403	0.103	-0.267	-0.224
infantMortality	-0.396	-0.023	0.282	0.200
GDPperCapita	0.233	0.682	-0.007	0.691
illiteracyMale	-0.341	0.493	-0.102	-0.408
illiteracyFemale	-0.353	0.479	-0.018	-0.320

```
for component in loadings.columns:
    print(component)
    print(loadings[component].sort_values(key=abs, ascending=False))
    print()
```

```
PC1
lifeFemale          0.402749
infantMortality    -0.396259
lifeMale            0.391031
tfr                -0.372029
illiteracyFemale   -0.353138
illiteracyMale     -0.340633
contraception       0.307135
GDPperCapita        0.233220
Name: PC1, dtype: float64

PC2
GDPperCapita        0.681981
illiteracyMale      0.493111
illiteracyFemale    0.478731
contraception       0.159688
```

```
lifeMale            0.153049
lifeFemale          0.103450
tfr                -0.048908
infantMortality    -0.023278
Name: PC2, dtype: float64

PC3
contraception       0.866758
lifeMale           -0.287134
infantMortality     0.281645
lifeFemale         -0.267079
illiteracyMale     -0.101806
tfr                -0.069985
illiteracyFemale   -0.017682
GDPperCapita       -0.007384
Name: PC3, dtype: float64

PC4
GDPperCapita        0.691463
illiteracyMale     -0.407970
illiteracyFemale   -0.320403
lifeMale           -0.261908
contraception      -0.255672
lifeFemale         -0.223798
infantMortality     0.200197
tfr                 0.169273
Name: PC4, dtype: float64
```

PC1 explains approximately 70.2% of the variance and represents a broad development and health dimension. Life expectancy and contraception are positioned in the opposite direction from fertility, infant mortality and illiteracy. PC2 explains approximately 12.6% and is influenced mainly by GDP per capita and literacy-related differences. Four components retain approximately 95.3% of the original information, so PCA provides meaningful dimensionality reduction.

```
k_values = range(2, 9)

inertias = []
silhouette_scores = []

for k in k_values:
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=50
    )

    labels = model.fit_predict(X_pca)

    inertias.append(model.inertia_)
    silhouette_scores.append(
        silhouette_score(X_pca, labels)
    )

evaluation = pd.DataFrame({
    "k": list(k_values),
    "Inertia": inertias,
    "Silhouette score": silhouette_scores
})

evaluation.round(3)
```

	k	Inertia	Silhouette score
0	2	674.837	0.520
1	3	493.846	0.396
2	4	387.381	0.365
3	5	328.348	0.329
4	6	290.490	0.294
5	7	261.031	0.313
6	8	233.569	0.305

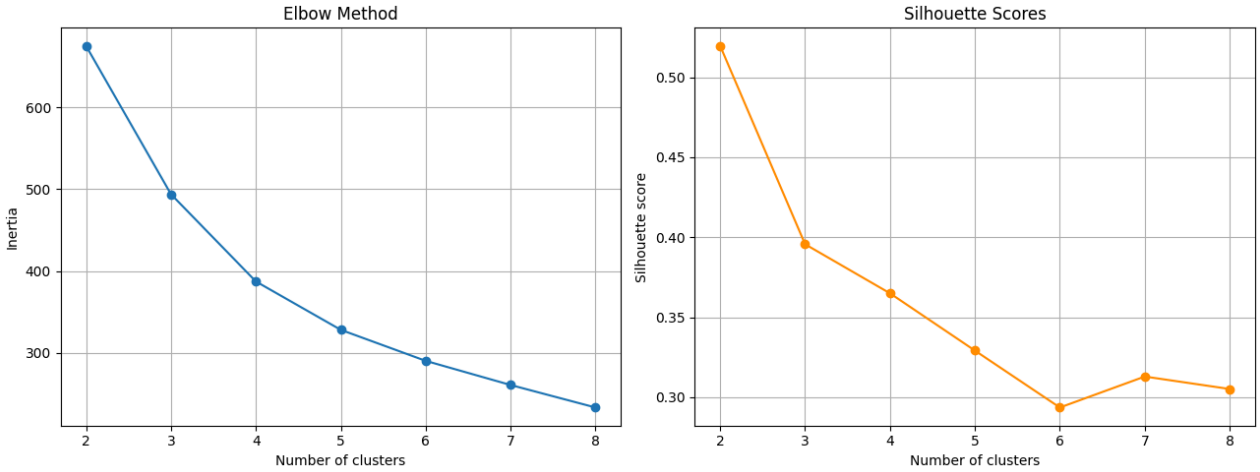
```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].plot(k_values, inertias, marker="o")
axes[0].set_title("Elbow Method")
axes[0].set_xlabel("Number of clusters")
axes[0].set_ylabel("Inertia")
```

```
axes[0].grid(True)

axes[1].plot(k_values, silhouette_scores, marker="o", color="darkorange")
axes[1].set_title("Silhouette Scores")
axes[1].set_xlabel("Number of clusters")
axes[1].set_ylabel("Silhouette score")
axes[1].grid(True)

plt.tight_layout()
plt.show()
```



The silhouette score is highest for two clusters, showing that the broad developed/developing division is visible in the data. However, the elbow curve continues to improve substantially until approximately four clusters. Four clusters reveal meaningful intermediate development groups that are hidden by the two-cluster solution. Therefore, four clusters were chosen as the preferred interpretation, while recognizing that two clusters provide the clearest statistical separation.

```
kmeans_2 = KMeans(
    n_clusters=2,
    random_state=42,
    n_init=50
)

kmeans_4 = KMeans(
    n_clusters=4,
    random_state=42,
    n_init=50
)

labels_2 = kmeans_2.fit_predict(X_pca)
labels_4 = kmeans_4.fit_predict(X_pca)
```

now we Compare two and four clusters

```
inertia_reduction = (
    (kmeans_2.inertia_ - kmeans_4.inertia_) /
    kmeans_2.inertia_
) * 100

round(inertia_reduction, 2)
```

```
42.6
```

Moving from two to four clusters reduces within-cluster variation by approximately 42.6%. This indicates that the four-cluster solution captures considerably more detail, although its silhouette score is lower.

```
results = df[["country", "region"]].copy()
results["cluster_2"] = labels_2
results["cluster_4"] = labels_4

results.head()
```

	country	region	cluster_2	cluster_4
0	Afghanistan	Asia	1	2
1	Albania	Europe	0	0
2	Algeria	Africa	0	0
3	American.Samoa	Asia	0	0
4	Andorra	Europe	0	0

```
for cluster in sorted(results["cluster_4"].unique()):
    print(f"\nCluster {cluster}")
    print(
        results.loc[
            results["cluster_4"] == cluster,
            "country"
        ].tolist()
    )
```

```
Cluster 0
['Albania', 'Algeria', 'American.Samoa', 'Andorra', 'Antigua', 'Argentina', 'Armenia', 'Azerbaijan', 'Bahrain', 'Barbados',

Cluster 1
['Angola', 'Bolivia', 'Botswana', 'Cambodia', 'Cameroon', 'Comoros', 'Congo', 'Dem.Rep.of.the.Congo', 'East.Timor', 'Egypt',

Cluster 2
['Afghanistan', 'Bangladesh', 'Benin', 'Bhutan', 'Burkina.Faso', 'Burundi', 'Central.African.Rep', 'Chad', 'Djibouti', 'Ethi

Cluster 3
['Australia', 'Austria', 'Bahamas', 'Belgium', 'Brunei', 'Canada', 'Denmark', 'Finland', 'France', 'French.Guiana', 'French.
```

```
cluster_profiles = X_imputed.copy()
cluster_profiles["cluster"] = labels_4

cluster_profiles.groupby("cluster").median().round(2)
```

cluster	tfr	contraception	lifeMale	lifeFemale	infantMortality	GDPperCapita	illiteracyMale	illiteracyFemale
0	2.66	47.0	67.50	73.75	23.0	1921.5	6.35	9.7
1	4.90	33.0	57.00	59.90	68.0	973.0	13.70	30.0
2	5.98	15.5	48.55	51.55	105.0	313.0	48.60	73.8
3	1.83	63.0	73.90	79.85	7.0	21472.0	10.83	20.1

The four clusters should contain approximately:

- 100 countries
- 37 countries
- 32 countries
- 38 countries The cluster numbers may appear in another order. That is normal.

```
representatives = []

for cluster in range(4):
    indices = np.where(labels_4 == cluster)[0]

    distances = np.linalg.norm(
        X_pca[indices] -
        kmeans_4.cluster_centers_[cluster],
        axis=1
    )

    representative_index = indices[np.argmin(distances)]

    representatives.append({
        "Cluster": cluster,
        "Country": df.loc[representative_index, "country"],
        "Region": df.loc[representative_index, "region"]
```

```
    })

representatives = pd.DataFrame(representatives)
```

	Cluster	Country	Region
0	0	Mexico	America
1	1	Namibia	Africa
2	2	Bhutan	Asia
3	3	Finland	Europe

With the specified settings, the representative countries should be close to:

- Mexico
- Namibia
- Bhutan
- Finland The cluster numbers may be reordered.

1. Can the world be divided into only two clusters? The PCA plot shows a clear general separation between two groups of countries. The silhouette score is 0.520, which means that the two clusters are reasonably well separated. This suggests that the traditional division between countries with stronger development conditions and countries with greater development challenges does appear in the data. However, the separation is not perfect. The countries are spread continuously across the graph, and some are located between the two main groups. This means that not every country can be clearly described as either developed or developing. Therefore, two clusters are useful for showing the main general difference between countries, but they are too simple to explain the complete structure of the world. More clusters may reveal the intermediate development levels hidden inside these two large groups. We will test this using the Elbow Rule, silhouette scores and additional cluster solutions.

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

kmeans_2 = KMeans(n_clusters=2, random_state=42, n_init=10)
clusters_2 = kmeans_2.fit_predict(X_pca)

print("Silhouette score for 2 clusters:",
      silhouette_score(X_pca, clusters_2))

plt.figure(figsize=(8, 6))
plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=clusters_2,
    cmap="viridis"
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Countries Divided into 2 Clusters")
plt.colorbar(label="Cluster")
plt.show()
```

```
Silhouette score for 2 clusters: 0.5196252546024205
```

